

3) Configuring Kubespray Inventory and Parameters

In this step, we configure the **'Inventory'** that defines the targets (node information) and methods (installation parameters) Ansible will use to build the cluster. All operations are performed within the `kubespray-2.29.0` directory on the Bastion server.

3.1. Installing Python Environment and Ansible Dependencies

(This assumes the Python virtual environment was activated in the previous step via the `source ~/.venv/3.11/bin/activate` command.)

```
# Navigate to the directory where the Kubespray playbook was extracted.
$ cd kubespray-2.29.0

# (Verify the virtual environment is active by checking for (3.11) in the prompt.)

# Upgrade pip to the latest version.
(3.11) $ pip install -U pip

# Install Ansible and related libraries required to run Kubespray.
# (These are installed via the offline YUM repository on the Bastion server.)
(3.11) $ pip install -r requirements.txt
```

3.2. Creating Cluster Inventory

Kubespray builds the cluster based on configuration files located inside the `inventory` directory. Copy the default `sample` inventory to create a new inventory for the cluster being built (e.g., `ssg`).

Bash

```
# Copy the 'sample' directory to a new directory named 'ssg'.
$ cp -rfp inventory/sample inventory/ssg
```

Before running the playbook, you must open the `inventory/ssg/inventory.ini` file and accurately enter the IP and connection information for each node that will constitute the K8s cluster under the `[all]`, `[kube_control_plane]`, `[kube_node]`, and `[etcd]` groups.

Bash

```
$ vim inventory/ssg/inventory.ini
```

Modification Guidelines:

- **[kube_control_plane]:** Modify Control Plane (Master Node) IPs (Update both `ansible_host` and `ip`).
- **[kube_node]:** Modify Worker Node IPs (Update both `ansible_host` and `ip`).
- **[etcd]:** Set etcd member names.

Example `inventory.ini` :

Ini, TOML

```
[sample]
# This inventory describes a HA topology with stacked etcd (= same nodes as control plane)
# and 3 worker nodes.
# See <https://docs.ansible.com/ansible/latest/inventory_guide/intro_inventory.html>
# for tips on building your inventory.

# Configure 'ip' variable to bind kubernetes services on a different ip than the default iface.
# We should set etcd_member_name for etcd cluster. The nodes that are not etcd members do not need to set the
value,
# or can set the empty string value.

[kube_control_plane]
node1 ansible_host=95.54.0.12 ip=10.3.0.1 etcd_member_name=etcd1
node2 ansible_host=95.54.0.13 ip=10.3.0.2 etcd_member_name=etcd2
node3 ansible_host=95.54.0.14 ip=10.3.0.3 etcd_member_name=etcd3

[etcd:children]
kube_control_plane

[kube_node]
node4 ansible_host=95.54.0.15 ip=10.3.0.4
node5 ansible_host=95.54.0.16 ip=10.3.0.5
node6 ansible_host=95.54.0.17 ip=10.3.0.6
```

3.3. Core Cluster Configuration (`k8s-cluster.yml`)

Modify the core configuration file for the created `ssg` inventory. This file defines key cluster behaviors such as CNI, DNS, and ownership.

Bash

```
$ vim inventory/ssg/group_vars/k8s_cluster/k8s-cluster.yml
```

Modify the file content according to the descriptions below.

3.3.1. Modifying Kube Owner (When using Cilium)

(line: 26) Change kube_owner to root. (This may be required to prevent permission issues when using Cilium CNI.)

YAML

```
kube_owner: root
```

Explanation: Cilium uses eBPF, which requires accessing and mounting the /sys/fs/bpf special file system. The Cilium agent (DaemonSet) runs in privileged: true mode to load eBPF programs into the kernel. Kubespray needs to perform preparatory work (like mounting /sys/fs/bpf) on the host node before launching this privileged container. This preparation is designed to be performed by kube_owner, hence root privileges are required.

Conclusion: Setting kube_owner: root is the standard deployment method in Kubespray to enable Cilium's eBPF features.

3.3.2. Changing CNI (Network Plugin)

(line: 67) Change Kubespray's default network plugin (calico) to cilium.

YAML

```
kube_network_plugin: cilium
```

3.3.3. kube_proxy ARP Configuration (MetalLB Compatibility)

(line: 124) Change the kube_proxy_strict_arp value to true.

YAML

```
# Preventing conflict with MetalLB: This is a mandatory prerequisite for using MetalLB (metallb_enabled: true).  
kube_proxy_strict_arp: true
```

3.3.4. Disabling NodeLocal DNS Cache

(line: 167) Disable the nodelocaldns feature by setting it to false.

YAML

```
enable_nodelocaldns: false
```

Reason: nodelocaldns is a performance optimization addon. However, in production environments, stability and simplicity are prioritized. Standard architecture allows the central CoreDNS to handle traffic, and scale-out can be achieved simply by increasing CoreDNS replicas if bottlenecks occur.

3.3.5. Enabling CoreDNS External Domain Resolution

(line: 195) Set to true to efficiently locate services exposed via MetalLB using their Internal IP even from within the cluster.

YAML

```
enable_coredns_k8s_external: true
```

Explanation: This implements a "Split-Horizon DNS" configuration to reduce unnecessary external network traffic (Hairpinning). Instead of routing traffic out to the external IP and back in, CoreDNS immediately returns the Internal IP assigned by MetalLB.

3.3.6. Fixing Cilium Offline Bugs

Add the following content to the bottom of the file. This resolves known offline bugs (Helm chart path, image name, Digest verification) that occur during Cilium installation.

YAML

```
# Specify Cilium Version
cilium_version: "1.18.2"

# Point Helm chart path to local file
cilium_helm_chart: "{{ files_repo }}/cilium-{{ cilium_version }}.tgz"

# Disable Private Registry Digest check and override Operator image path
cilium_extra_values:
  image:
    useDigest: false
  hubble:
    relay:
      image:
        useDigest: false
  ui:
    backend:
      image:
        useDigest: false
    frontend:
      image:
        useDigest: false
  operator:
    image:
      override: "{{ registry_host }}/cilium/operator-generic:v1.18.2"
      useDigest: false
    extension: ""
  certgen:
    image:
      useDigest: false
  envoy:
    image:
      useDigest: false
```

3.4. Addons Configuration (`addons.yml`)

Configure additional Kubernetes cluster features. Here, we enable MetalLB to implement LoadBalancer services.

Bash

```
$ vim inventory/ssg/group_vars/k8s_cluster/addons.yml
```

(line: 147) Change `metallb_enabled` from `false` to `true` .

YAML

```
metallb_enabled: true
```

Explanation: When creating a service of type: LoadBalancer, MetalLB allocates an IP from the configured range (same subnet as K8s nodes), allowing the service to communicate with the external network.

3.5. Offline Repository Configuration (`offline.yml`)

Configure the Bastion server's repository address to be referenced by all nodes (`all`). This file is critical for air-gapped installation.

Bash

```
$ vim inventory/ssg/group_vars/all/offline.yml
```

Overwrite the `en13tire` file content with the settings below.

[Action Required] You must accurately change the `[IP]` part to the actual IP address of the Bastion server (e.g., 172.30.31.14).

YAML

```
http_server: "http://[IP]" # Server address hosting the private registry/files
registry_host: "[IP]:35000"
```

```
# Insecure registries for containerd
containerd_registries_mirrors:
- prefix: "{{ registry_host }}"
  mirrors:
  - host: "http://{{ registry_host }}"
    capabilities: ["pull", "resolve"]
    skip_verify: true
- prefix: docker.io
  mirrors:
  - host: "http://{{ registry_host }}"
    capabilities: ["pull", "resolve"]
    skip_verify: true
- prefix: ghcr.io
  mirrors:
  - host: "http://{{ registry_host }}"
```

```

    capabilities: ["pull", "resolve"]
    skip_verify: true
- prefix: gcr.io
  mirrors:
    - host: "http://{{ registry_host }}"
      capabilities: ["pull", "resolve"]
      skip_verify: true
- prefix: quay.io
  mirrors:
    - host: "http://{{ registry_host }}"
      capabilities: ["pull", "resolve"]
      skip_verify: true
- prefix: k8s.gcr.io
  mirrors:
    - host: "http://{{ registry_host }}"
      capabilities: ["pull", "resolve"]
      skip_verify: true
- prefix: registry.k8s.io
  mirrors:
    - host: "http://{{ registry_host }}"
      capabilities: ["pull", "resolve"]
      skip_verify: true

files_repo: "{{ http_server }}/files"
yum_repo: "{{ http_server }}/rpms"
ubuntu_repo: "{{ http_server }}/debs"

# Registry overrides
kube_image_repo: "{{ registry_host }}"
gcr_image_repo: "{{ registry_host }}"
docker_image_repo: "{{ registry_host }}"
quay_image_repo: "{{ registry_host }}"

# Download URLs: See roles/download/defaults/main.yml of kubespray.
kubeadm_download_url: "{{ files_repo }}/kubernetes/v{{ kube_version }}/kubeadm"
kubectrl_download_url: "{{ files_repo }}/kubernetes/v{{ kube_version }}/kubectrl"
kubectl_download_url: "{{ files_repo }}/kubernetes/v{{ kube_version }}/kubectl"
kubectl_download_url: "{{ files_repo }}/kubernetes/v{{ kube_version }}/kubectl"

# etcd is optional if you **DON'T** use etcd_deployment=host
etcd_download_url: "{{ files_repo }}/kubernetes/etcd/etcd-v{{ etcd_version }}-linux-amd64.tar.gz"
cni_download_url: "{{ files_repo }}/kubernetes/cni/cni-plugins-linux-{{ image_arch }}-v{{ cni_version }}.tgz"
crictl_download_url: "{{ files_repo }}/kubernetes/crictl-v{{ crictl_version }}-{{ ansible_system | lower }}-{{ image_arch }}.tar.gz"

# If using Calico
calicoctl_download_url: "{{ files_repo }}/kubernetes/calico/v{{ calico_ctl_version }}/calicoctl-linux-{{ image_arch }}"
# If using Calico with kdd
calico_crds_download_url: "{{ files_repo }}/kubernetes/calico/v{{ calico_version }}.tar.gz"

runc_download_url: "{{ files_repo }}/runc/v{{ runc_version }}/runc.{{ image_arch }}"
nerdctl_download_url: "{{ files_repo }}/nerdctl-{{ nerdctl_version }}-{{ ansible_system | lower }}-{{ image_arch }}.tar.gz"
containerd_download_url: "{{ files_repo }}/containerd-{{ containerd_version }}-linux-{{ image_arch }}.tar.gz"

# Fix Cilium CLI binary download bug (Issue #12558)

```

```
ciliumcli_download_url: "{{ files_repo }}/cilium-cli/v0.18.7/cilium-linux-amd64.tar.gz"
ciliumcli_checksum: ""
```